

Project Management Plan: E-Commerce Application

Project Title: Flask-Based Online Shopping Application

Project Manager: Tom Cruise

Timeline: January 2026 – December 2026 (12 Months)

Tech Stack: Python, Flask, PostgreSQL, Docker, Jinja2, TailwindCSS

1. Executive Summary

The purpose of this project is to build and deploy a secure, lightweight, and highly scalable **Online Shopping Application** utilizing the **Python Flask framework**. Initiated in January 2026 and slated for production launch in December 2026, the application targets modern e-commerce standards, offering a responsive interface, efficient session handling, and structured persistent storage.

By leveraging Python's robust ecosystem alongside a decoupled relational database structure management strategy, the platform ensures rapid execution speeds and highly reliable transaction management. The project plan outlines the systems engineering pipeline, deployment architecture, multi-phase roadmap, and validation boundaries necessary to deliver the application within schedule and budget.

2. System Architecture & Data Flow

The platform executes via a classic **Model-View-Controller (MVC)** paradigm optimized for Flask blueprints. Front-end server-side rendering is handled natively through the Jinja2 template engine, integrated with TailwindCSS. Backend logic routes securely through Python runtime layers to an isolated PostgreSQL instance.

Data Flow Architecture Pipeline

Step 1: Client Request Layer

The client interacts via HTTP/S over TLS, hitting an Nginx reverse proxy which directly forwards appropriate upstream context to a Gunicorn WSGI server running the Flask core.

Step 2: Controller & Session Validation

Flask Application Blueprints ingest request variables. Session integrity is checked natively using cryptographically signed client-side cookies or server-side Redis structures to avoid unauthorized data mutations.

Step 3: Database & Model Mapping (SQLAlchemy ORM)

Data operations interface through SQLAlchemy ORM. Transactions follow ACID properties rigorously. Database connection polling is tuned through a localized connection pool mapped to PostgreSQL container layers.

3. Implementation Roadmap & Schedule

The 12-month timeline is broken down into structured iterative phases, balancing safe prototyping loops with aggressive automated validation cycles.

Phase & Timeline	Core Objectives & Activities	Key Deliverables
Phase 1: Initiation Jan – Feb 2026	Requirements discovery, UI/UX framing, schema formulation, and architectural baselining.	Software Requirements Spec (SRS), Database ERD, Git Repository Setup.
Phase 2: Core Engineering Mar – Jun 2026	Implementing core Flask blueprints, authentication (Flask-Login), product catalogs, and cart micro-logic.	Functional MVC Core, Local Database Migrations (Alembic).
Phase 3: Integration Jul – Aug 2026	Integration of payment mock gateways, order placement pipelines, and automated notification setups.	Stripe Sandbox Integration, Transaction Logging System.
Phase 4: Optimization Sep – Oct 2026	Performance bottleneck analysis, query optimizations, Docker containerization, and staging setups.	Docker Compose configurations, Load testing metrics sheets.
Phase 5: Release Ops Nov – Dec 2026	User Acceptance Testing (UAT), final production data seeding, CI/CD pipeline confirmation, and formal deployment.	Live Application Instance, Handover documentation, Final Project Sign-off.

4. Resource & Hardware Requirements

Human Resource Allocations

- **Project Manager / Lead Engineer (Tom Cruise):** Responsible for timeline tracking, architecture validation, and operational deployment.
- **Full-Stack Engineers (2):** Tasked with UI execution, Flask routing, and backend optimization workflows.
- **QA / Validation Engineer (1):** Focused on automated unit tests, transaction simulation, and security patching.

Staging & Production Infrastructure Topology

To support high concurrent transaction ceilings, the system maps out specific isolated resource configurations:

Infrastructure Layer	Hardware Specification (Minimum)	Software Environment Roles
Application Node	Virtual Instance, 2 vCPUs, 4GB RAM, 50GB SSD	Ubuntu Server OS, Docker Engine, Python 3.11+, Gunicorn
Database Instance	Managed Relational Node, 2 vCPUs, 8GB RAM, 100GB NVMe	PostgreSQL 15+, Automated daily snapshots enabled
Caching Engine	In-Memory Node, 1GB RAM	Redis (Session store management & Rate limiting)

5. Risk Management & Validation Constraints

Project mitigation requires keeping explicit boundary metrics on failure vectors. To quantify software load handling limitations and database safety, the following operational constraints are strictly applied.

Quantifiable System Constraints

The platform response latency target (L_p) during transactional checkout surges must satisfy the following calculation where S_n is system server network overhead and D_q is the database execution queue length:

$$L_p = (S_n \times 1.5) + \sqrt{(D_q)} \leq 200ms$$

If at any point during load testing calculations exceed this constraint threshold, systemic connection pooling optimization is automatically triggered to prevent application timeouts.

Critical Security Rule: All input vectors originating from client requests must undergo rigorous verification using WTForms / Marshmallow schemas to completely prevent SQL injection attacks and Cross-Site Scripting (XSS) issues in Jinja templates.

Risk Evaluation Table

Identified Risk	Probability / Impact	Proactive Mitigation Strategy
Database Concurrency Bottleneck	Medium / High	Implement strict connection pooling bounds via SQLAlchemy and enable structural Redis caching for highly read-heavy catalog operations.
Scope Creep via Payment Logic	Low / Medium	Restrict scope strictly to standard checkout protocols utilizing simple webhooks; push custom enterprise features out to a post-December 2026 release schedule.
Data Inconsistency Issues	Low / High	Enforce atomic database transactions explicitly using context managers (<i>db.session.begin_nested()</i>) ensuring rollbacks occur natively on any operational fault.